شرط‌ها و منطق تصمیم‌گیری در پایتون

برنامه‌ها همیشه مجموعه‌ای از دستورهای ثابت نیستند. بسیاری از اوقات لازم است برنامه با توجه به اطلاعات ورودی، وضعیت فعلی یا نتیجه یک محاسبه، تصمیم متفاوتی بگیرد.

برای مثال، یک برنامه ممکن است بررسی کند که آیا کاربر رمز عبور درستی وارد کرده است، آیا موجودی حساب برای خرید کافی است یا آیا نمره دانشجو به حد قبولی رسیده است.

در پایتون، برای پیاده‌سازی چنین تصمیم‌هایی از شرط‌ها و عملگرهای منطقی استفاده می‌کنیم.

۱. دستور `if`

با استفاده از `if` می‌توانیم مشخص کنیم که یک بخش از برنامه فقط در صورت برقرار بودن یک شرط اجرا شود.

برای مثال:

```
age = 20

if age >= 18:
    print("You are an adult.")
```

در این برنامه، شرط `age >= 18` بررسی می‌کند که آیا سن حداقل ۱۸ سال است یا خیر. چون شرط برقرار است، پیام نمایش داده می‌شود.

ساختار کلی شرط در پایتون به این صورت است:

```
if condition:
    # Code to execute when the condition is true
```

به دو نکته توجه کنید:

- بعد از شرط، علامت : قرار می‌گیرد.
- دستورهای داخل شرط باید با تورفتگی نوشته شوند. این تورفتگی در پایتون بخشی از ساختار برنامه است و معمولاً از چهار فاصله استفاده می‌کنیم.

اگر شرط برقرار نباشد، دستورهای داخل آن اجرا نمی‌شوند.

۲. استفاده از `else`

گاهی لازم است برای هر دو حالتِ برقرار بودن و برقرار نبودن شرط، رفتاری مشخص کنیم. در این حالت از `else` استفاده می‌کنیم.

```
age = 16

if age >= 18:
    print("You are an adult.")
else:
    print("You are under 18.")
```

خروجی:

```
You are under 18.
```

اگر شرط `if` برقرار باشد، بخش اول اجرا می‌شود؛ در غیر این صورت، بخش `else` اجرا خواهد شد.

۳. بررسی چند حالت با `elif`

گاهی دو حالت کافی نیست و باید چند وضعیت مختلف را بررسی کنیم. برای این کار از `elif` استفاده می‌کنیم.

برای مثال، می‌خواهیم بر اساس نمره دانشجو، وضعیت او را مشخص کنیم:

```
score = 75

if score >= 90:
    print("Excellent")
elif score >= 70:
    print("Good")
elif score >= 50:
    print("Passed")
else:
    print("Failed")
```

خروجی:

```
Good
```

پایتون شرط‌ها را از بالا به پایین بررسی می‌کند. به‌محض اینکه یکی از شرط‌ها برقرار باشد، بخش مربوط به آن اجرا می‌شود و بقیه شرط‌های همین زنجیره بررسی نمی‌شوند.

به همین دلیل، ترتیب شرط‌ها اهمیت دارد. برای مثال، اگر شرط `score >= 50` را در ابتدای این زنجیره قرار دهیم، نمره 75 نیز همان‌جا پذیرفته می‌شود و بخش‌های بعدی بررسی نخواهند شد.

۴. عملگرهای مقایسه

برای تصمیم‌گیری، معمولاً باید دو مقدار را با هم مقایسه کنیم. پایتون عملگرهای زیر را در اختیار ما قرار می‌دهد:

عملگر	مفهوم	مثال
`==`	بودن برابر	`age == 18`
`!=`	بودن نابرابر	`age != 18`
`>`	بودن تربزرگ	`age > 18`
`<`	بودن ترکوچک	`age < 18`
`>=`	بودن مساوی یا تربزرگ	`score >= 50`
`<=`	بودن مساوی یا ترکوچک	`price <= 100`

نتیجه این مقایسه‌ها معمولاً یکی از دو مقدار منطقی `True` یا `False` است.

برای مثال:

```
price = 80

print(price < 100)
print(price == 100)
print(price != 100)
```

خروجی:

```
True
False
True
```

نکته مهم: علامت = برای انتساب مقدار به متغیر است، اما == برای مقایسه دو مقدار استفاده می‌شود.

```
age = 18        # Assign a value
print(age == 18)  # Compare two values
```

۵. ترکیب شرط‌ها با `and` و `or`

گاهی یک تصمیم به بیش از یک شرط وابسته است.

عملگر `and`

عملگر `and` زمانی نتیجه درست می‌دهد که تمام شرط‌ها درست باشند.

برای مثال، فرض کنید خرید فقط برای کاربران ثبت‌نام‌شده‌ای مجاز است که موجودی کافی دارند:

```
is_registered = True
has_enough_balance = True

if is_registered and has_enough_balance:
    print("Purchase allowed.")
else:
    print("Purchase not allowed.")
```

در این مثال، هر دو شرط باید برقرار باشند تا پیام تأیید خرید نمایش داده شود.

عملگر `or`

عملگر `or` زمانی نتیجه درست می‌دهد که حداقل یکی از شرط‌ها درست باشد.

برای مثال، ممکن است تخفیف به دانشجویان یا افراد بالای ۶۰ سال تعلق بگیرد:

```
is_student = False
age = 65

if is_student or age > 60:
    print("Discount available.")
else:
    print("No discount.")
```

چون سن بیشتر از ۶۰ سال است، شرط برقرار خواهد بود.

عملگر `not`

عملگر `not` نتیجه منطقی را برعکس می‌کند؛ یعنی `True` را به `False` و `False` را به `True` تبدیل می‌کند.

```
is_blocked = False

if not is_blocked:
    print("Access allowed.")
```

چون `is_blocked` برابر `False` است، عبارت `not is_blocked` درست خواهد بود.

۶. بررسی عضویت با `in`

گاهی می‌خواهیم بررسی کنیم که یک مقدار در یک مجموعه از داده‌ها وجود دارد یا خیر. برای این کار می‌توانیم از `in` استفاده کنیم.

```
allowed_users = ["Sara", "Ali", "Reza"]
user_name = "Ali"

if user_name in allowed_users:
    print("Access allowed.")
else:
    print("Access denied.")
```

خروجی:

```
Access allowed.
```

در این مثال، برنامه بررسی می‌کند که آیا نام کاربر در فهرست کاربران مجاز وجود دارد یا خیر.

این روش برای بررسی وجود یک مقدار در لیست، رشته، مجموعه و برخی ساختارهای داده دیگر کاربرد دارد.

۷. استفاده از ورودی کاربر در تصمیم‌گیری

اکنون می‌توانیم شرط‌ها را با مفاهیم قبلی، مانند متغیرها و ورودی کاربر، ترکیب کنیم.

فرض کنید می‌خواهیم برنامه‌ای بنویسیم که سن کاربر را دریافت کند و وضعیت او را نمایش دهد:

```
age = int(input("Enter your age: "))

if age < 0:
    print("Invalid age.")
elif age < 18:
    print("You are under 18.")
else:
    print("You are 18 or older.")
```

در این مثال:

1. برنامه سن را از کاربر دریافت می‌کند.
2. با `int()` ورودی را به عدد صحیح تبدیل می‌کند.
3. ابتدا بررسی می‌کند که سن منفی نباشد.
4. سپس مشخص می‌کند که سن کمتر از ۱۸ سال است یا خیر.
5. در نهایت، پیام مناسب را نمایش می‌دهد.

البته اگر کاربر به‌جای عدد، متنی نامعتبر وارد کند، تبدیل به عدد خطا ایجاد می‌کند. مدیریت این نوع خطاها را می‌توانیم در مبحث مدیریت خطاها بررسی کنیم.

۸. منطق تصمیم‌گیری در برنامه‌های واقعی

شرط‌ها زمانی ارزش بیشتری پیدا می‌کنند که بخشی از یک مسئله واقعی باشند.

برای مثال، در برنامه‌ای که مبلغ خرید را محاسبه می‌کند، می‌توانیم برای خریدهای بالاتر از یک مقدار مشخص تخفیف در نظر بگیریم:

```
total_price = 150

if total_price >= 100:
    discount = total_price * 0.10
else:
    discount = 0

final_price = total_price - discount

print(f"Final price: {final_price}")
```

خروجی:

```
Final price: 135.0
```

در این برنامه، ابتدا مبلغ خرید بررسی می‌شود. سپس متناسب با نتیجه شرط، میزان تخفیف تعیین و قیمت نهایی محاسبه می‌شود.

این مثال ساده نشان می‌دهد که برنامه‌نویسی تنها اجرای دستورها نیست؛ بلکه تبدیل قوانین و تصمیم‌های یک مسئله به دستورهای قابل اجراست.

۹. اهمیت منطق تصمیم‌گیری هنگام استفاده از هوش مصنوعی

هوش مصنوعی می‌تواند برای ما شرط‌ها و منطق برنامه را بنویسد، اما ممکن است شرطی را ناقص یا اشتباه پیاده‌سازی کند.

برای نمونه، در یک برنامه محاسبه تخفیف باید مشخص باشد که تخفیف برای خریدهای بیشتر از ۱۰۰ واحد اعمال می‌شود یا برای خریدهای برابر با ۱۰۰ واحد نیز در نظر گرفته شده است. تفاوت میان < و =< در اینجا می‌تواند نتیجه را تغییر دهد.

هنگام بررسی کدهای تولیدشده با هوش مصنوعی، به این پرسش‌ها توجه کنید:

- آیا همه حالت‌های مهم مسئله بررسی شده‌اند؟
- آیا شرط‌ها به ترتیب مناسبی قرار گرفته‌اند؟
- آیا از `and` و `or` به‌درستی استفاده شده است؟
- اگر شرط برقرار نباشد، برنامه چه رفتاری دارد؟
- آیا حالت‌های مرزی، مانند صفر یا مقدار دقیقاً برابر با حد تعیین‌شده، بررسی شده‌اند؟

هدف، حفظ‌کردن دستورها نیست؛ مهم این است که بتوانیم قوانین مسئله را به‌روشنی بیان کنیم و درستی پیاده‌سازی آن‌ها را بررسی کنیم.

جمع‌بندی

با استفاده از شرط‌های `if`، `elif` و `else` می‌توانیم مسیر اجرای برنامه را بر اساس وضعیت داده‌ها تغییر دهیم. عملگرهای مقایسه برای مقایسه مقادیر و عملگرهای منطقی مانند `and`، `or` و `not` برای ترکیب یا معکوس‌کردن شرط‌ها به کار می‌روند. همچنین `in` امکان بررسی عضویت یک مقدار در مجموعه‌ای از داده‌ها را فراهم می‌کند.

درک منطق تصمیم‌گیری به ما کمک می‌کند برنامه‌هایی کاربردی‌تر بسازیم و کدهایی را که خودمان یا هوش مصنوعی تولید کرده‌ایم، بهتر بررسی کنیم.

تمرین

برنامه‌ای بنویسید که نمره دانشجو را از کاربر دریافت کند و بر اساس مقدار آن، وضعیت دانشجو را نمایش دهد.

قوانین برنامه:

- اگر نمره کمتر از صفر یا بیشتر از ۲۰ باشد، پیام `score Invalid` نمایش داده شود.
- اگر نمره از ۱۷ بیشتر یا مساوی باشد، پیام `Excellent` نمایش داده شود.
- اگر نمره از ۱۰ بیشتر یا مساوی باشد، پیام `Passed` نمایش داده شود.
- در سایر حالت‌ها، پیام `Failed` نمایش داده شود.

نکته: شرط‌ها را به ترتیبی بنویسید که ابتدا نامعتبر بودن نمره بررسی شود. سپس حالت‌های معتبر را از نمره بالاتر به پایین‌تر بررسی کنید.